4. Confusion Matrix

Beyond Accuracy - The Truth About Performance

Previous Section:

3. One-Hot Encoding

Contents:

When studying Machine Learning, there is always this cycle: you train the model, and then you evaluate it. And while most learners are busy diving deep into the mathematics behind each model, they often overlook something just as important—the evaluation process itself.

Because at the end of the day, what's the point of a model if you don't know how well it actually performs? So what do we do?

- We take our validation set, or testing set—basically unseen data—and let the model make predictions.
- Then we measure how good those predictions are.

Usually, people jump straight to **Accuracy**. But hold on—how is accuracy even calculated?

And more importantly, is it the only thing we should care about? **ABSOLUTELY NOT**

Because today, it's not just about building models—it's about understanding **metrics**. And every single one of those metrics comes from one place. A small, ugly, but incredibly powerful table → The **Confusion Matrix**

1. Confusion Matrix - Positives and Negatives

A Confusion Matrix, is exactly what it sounds like—a matrix. But more importantly, Confusion Matrix is a tool used in **classification** to evaluate how well a model is performing, by directly comparing what the model **predicted** against what the **actual labels** are.

So instead of just giving you a single number like accuracy, it shows you the *full picture*—where the model is right, and more importantly... where it goes wrong.

Now, with that idea in mind, let's break down the structure:

- First of all: The confusion matrix is always **square**. Why? Because the number of predicted classes must match the number of actual classes.
- Second, and this is very important: The Confusion Matrix is used **only for classification problems**. Regression doesn't have "classes", so there is nothing to "confuse."

Now, let's start with the simplest form of a Confusion Matrix—the one used in **Binary Classification**. Since we only have two classes, the matrix is always **2 × 2**. And to keep everything consistent, we usually define:

- **Positive class → 1**
- **Negative class → 0**

Simple, clean, no ambiguity.

Inside this 2 × 2 matrix, there are four fundamental components—and this is where things start to matter:

- **True Positive (TP):** The model predicted *positive*, and it is actually *positive*.
- **True Negative (TN):** The model predicted *negative*, and it is actually *negative*.

So far, so good, these are the correct predictions. But here's where things get interesting:

- **False Positive (FP):** The model predicted *positive*, but it is actually *negative*.

  (Also known as Type I Error — the model is "too optimistic")

- **False Negative (FN):** The model predicted *negative*, but it is actually *positive*.

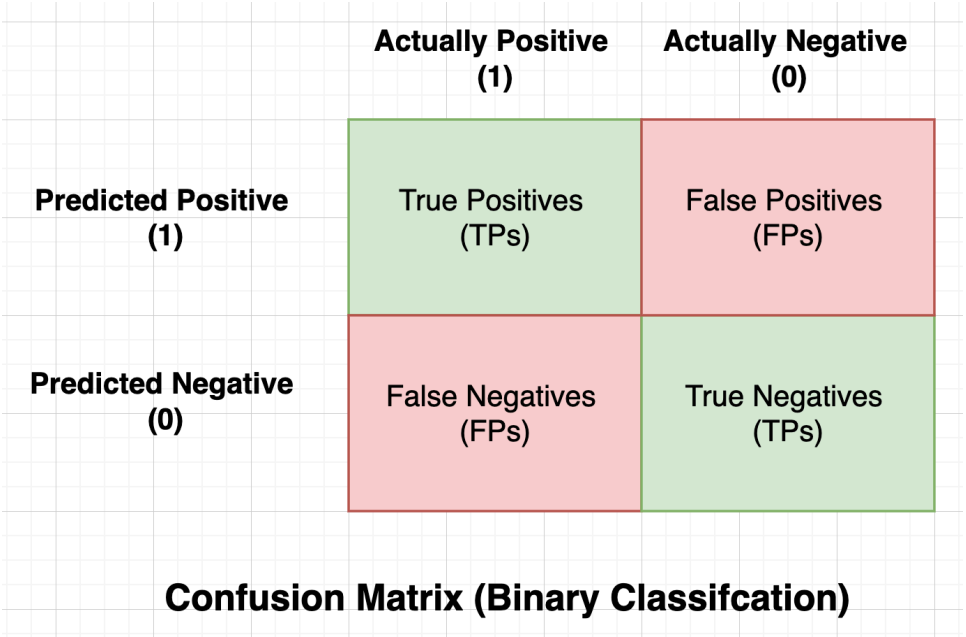
  (Also known as Type II Error — the model "misses" something important)

Now, one important note about how we represent this matrix: **You can technically arrange actual values and predicted values either as rows or column. Both are valid.**

But to stay consistent with most lectures and materials, we will follow this convention:

- **Columns → Actual values**
- **Rows → Predicted values**

Reference the following image:



And once you understand these four components, you've basically unlocked *every evaluation metric* that comes next.

2. Confusion Matrix - Evaluation Metrics

Now, when we talk about *evaluation metrics*, we usually mean metrics for both classification and regression. But here, we are strictly in the world of **classification**. So everything we measure will come directly from the Confusion Matrix.

First, a very simple but important question: **How many predictions did the model make in total?** It's just the sum of everything:

$$\text{Total Predictions} = TP + TN + FP + FN$$

That's your entire dataset used for predictions—nothing more, nothing less. Now, from these four values, we can derive the most commonly used evaluation metrics. And this is where the Confusion Matrix becomes *ridiculously powerful*.

There are five key metrics you'll see everywhere:

- **Accuracy:** How often is the model correct overall?

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Out of all predicted positives... how many are actually correct?

$$Precision = \frac{TP}{TP + FP}$$

- **Recall (or Sensitivity):** Out of all actual positives, how many did the model correctly identify?

$$Recall = \frac{TP}{TP + FN}$$

- **Specificity:** Out of all actual negatives, how many did the model correctly identify?

$$Specificity = \frac{TN}{TN + FP}$$

- **F1-Score:** A balance between Precision and Recall.

$$F1 = \frac{2 \times (Precision \times Recall)}{Precision + Recall}$$

And this is the key idea I want you to see: Every single one of these metrics... comes from just four numbers: TP, TN, FP, FN. That "Annoying Confusion Matrix" we talked about earlier? This is why it matters.

The following Python code is how you construct a Confusion Matrix completely **by hand** for binary classification.

And yes, this is the moment where things *click*.

```
import numpy as np
import pandas as pd

# A is for positive and B is for Negative
true_value = np.array(['A', 'B', 'A', 'B', 'B', 'B', 'A',
'A', 'A', 'B', 'B', 'A', 'B', 'B', 'B'])
predict_value = np.array(['A', 'A', 'B', 'A', 'B', 'B',
'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'B'])

# Replace the categoriy to numerical
true_value = np.where(true_value == 'B', 1, 0)
predict_value = np.where(predict_value == 'B', 1, 0)
# Make sure the positive is at the front by reversing
unique_labels = np.unique(true_value)[::-1]
```

```
# Construct the binary confusion matrix by hand
def get_confusion_matrix(y_true, y_pred, labels):
  confusion_matrix = np.zeros((len(labels), len(labels)))
  # True Positives (TPs) and True Negatives (TNs)
  for i in range(len(labels)):
    for j in range(len(y_true)):
      if y_true[j] == labels[i] and y_pred[j] == labels[i]:
        confusion_matrix[i, i] += 1
      # False Positives (FPs)
      elif y_true[j] != labels[i] and y_pred[j] == labels
[i]:
        # If the true_label is Positive (1) -> unique_label
s[0]
        # False Negatives (FNs)
        if labels[i] == unique_labels[0]:
          confusion_matrix[0, 1] += 1
        else:  # False Positives (FPs)
          confusion_matrix[1, 0] += 1

  return confusion_matrix

get_confusion_matrix(true_value, predict_value, unique_labe
ls)
```

```
array([[3., 1.],
    [6., 5.]])
```

Now, let me walk you through what's really happening inside this function.

First, notice this line:

- We define `unique_labels = np.unique(true_value)[::-1]`

This is not random. We are **forcing the positive class (1) to come first**, followed by the negative class (0).

That means our matrix will always follow a consistent structure:

- Row 0 / Column 0 → Positive class
- Row 1 / Column 1 → Negative class

This consistency is *extremely important*—because if you mess up the order, your entire interpretation of TP, FP, FN, TN will be flipped.

Now inside the function: We initialize an empty **2 × 2 matrix** filled with zeros.

Then we loop through:

- Each label (positive and negative)
- Each data point in the dataset

And for every single prediction, we manually check:

Case 1: Correct predictions

```
if y_true[j] == labels[i] and y_pred[j] == labels[i]:
        confusion_matrix[i, i] += 1
```

This fills the diagonal:

- Top-left → True Positive
- Bottom-right → True Negative

Case 2: Wrong predictions

```
elif y_true[j] != labels[i] and y_pred[j] == labels[i]:
```

Now we split into two situations:

- If we are dealing with the **positive class (1), t**his becomes a **False Negative (FN)**

  → We update: `confusion_matrix[0, 1]`
- Otherwise (negative class), this becomes a **False Positive (FP)**

  → We update: `confusion_matrix[1, 0]`

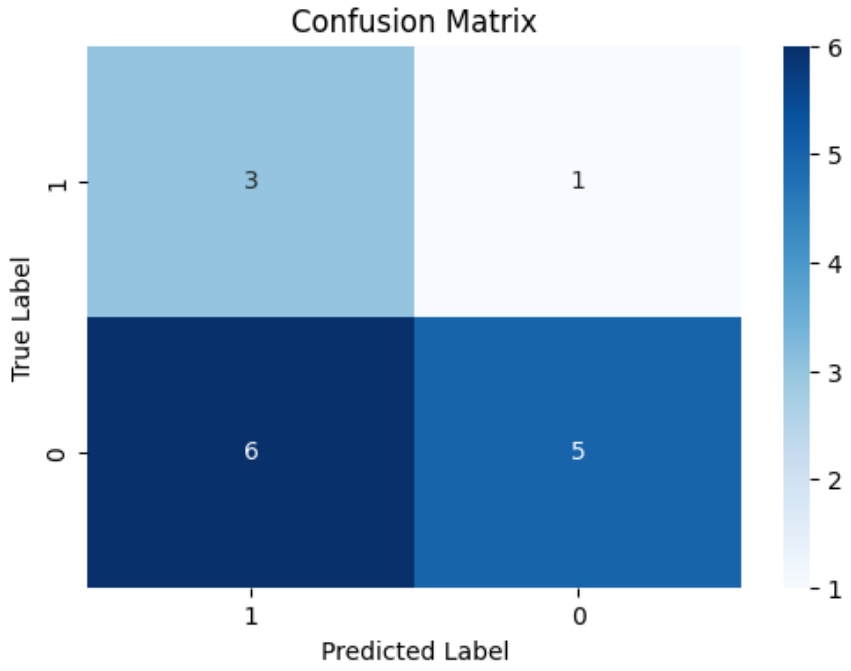
And that's it. No libraries. No shortcuts. Just pure logic.

Plot the confusion matrix:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot
confusion_matrix = get_confusion_matrix(true_value, predict
_value, unique_labels)
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix, annot=True, cmap="Blues",
```

```
            xticklabels=unique_labels, yticklabels=unique_l
abels)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



And this is exactly what I want you to realize, that “confusion matrix” people casually call from a library, is literally just: counting matches, counting mismatches, and putting them in the right position.

Once you understand this, you’re no longer just casually *using* Machine Learning tools. You’re actually **understanding them**.

Now, this is where everything comes together. We’ve built the Confusion Matrix by hand. And from that, we extract:

```
# Evaluation metrics
conf_matrix = get_confusion_matrix(true_value, predict_valu
e, unique_labels)
TP = conf_matrix[0, 0]
FP = conf_matrix[0, 1]
FN = conf_matrix[1, 0]
TN = conf_matrix[1, 1]
print(f"TP = {TP}; TN = {TN}; FP = {FP}; FN = {FN}")
print(f'Accuracy: {(TP + TN) / (TP + FP + FN + TN)}')
print(f'Precision: {TP / (TP + FP)}')
print(f'Recall: {TP / (TP + FN)}')
print(f'Specificity: {TN / (TN + FP)}')
print(f'F1 Score: {2 * TP / (2 * TP + FP + FN)}')
```

```
TP = 3.0; TN = 5.0; FP = 1.0; FN = 6.0
Accuracy: 0.5333333333333333
Precision: 0.75
Recall: 0.3333333333333333
Specificity: 0.8333333333333334
F1 Score: 0.46153846153846156
```

- $TP = 3, TN = 5, FP = 1, FN = 6$

  (Yes, they appear as floats, but conceptually, they are just counts.)

Now look at the metrics:

- $Accuracy \approx 0.53$
- $Precision = 0.75$
- $Recall \approx 0.33$
- $Specificity \approx 0.83$
- $F1Score \approx 0.46$

And this is exactly why Accuracy alone is *dangerous*. At first glance, 53% accuracy might make you think: ***"Okay... not great, not terrible."***

But look deeper:

- **Precision is high (0.75)** → When the model says *positive*, it's usually correct

- **Recall is very low (0.33)** → But it misses *a lot* of actual positives

That means this model is **conservative**: It doesn't predict positive often. But when it does, it's usually right.

Now flip to the other side:

- **Specificity is high (0.83)** → It's very good at identifying negatives

So overall, this model is good at avoiding false alarms but bad at catching important positive cases.

And that leads to the most important question: **Is this model actually good?** Well, it depends on the problem.

- If this is **spam detection** → maybe acceptable
- If this is **disease detection** → this is terrible (because missing positives = missing sick patients)

And this is the whole point of this section: The Confusion Matrix doesn't just give you numbers. It tells you a **story** about your model's behavior. Not just *how often* it is correct, but *how* it is correct—and more importantly, **how it fails.**

3. Confusion Matrix for Multi-Class Classification

So far, everything we've seen works nicely for **binary classification**. But reality? It's rarely that simple. Most problems don't have just *two* classes. They have three, five, sometimes dozens.

So what happens to the Confusion Matrix? It doesn't change its nature → It simply **scales up**.

In **Multi-Class Classification**, the confusion matrix becomes a larger **square matrix**, where:

- Each **row** represents the **actual class**
- Each **column** represents the **predicted class**

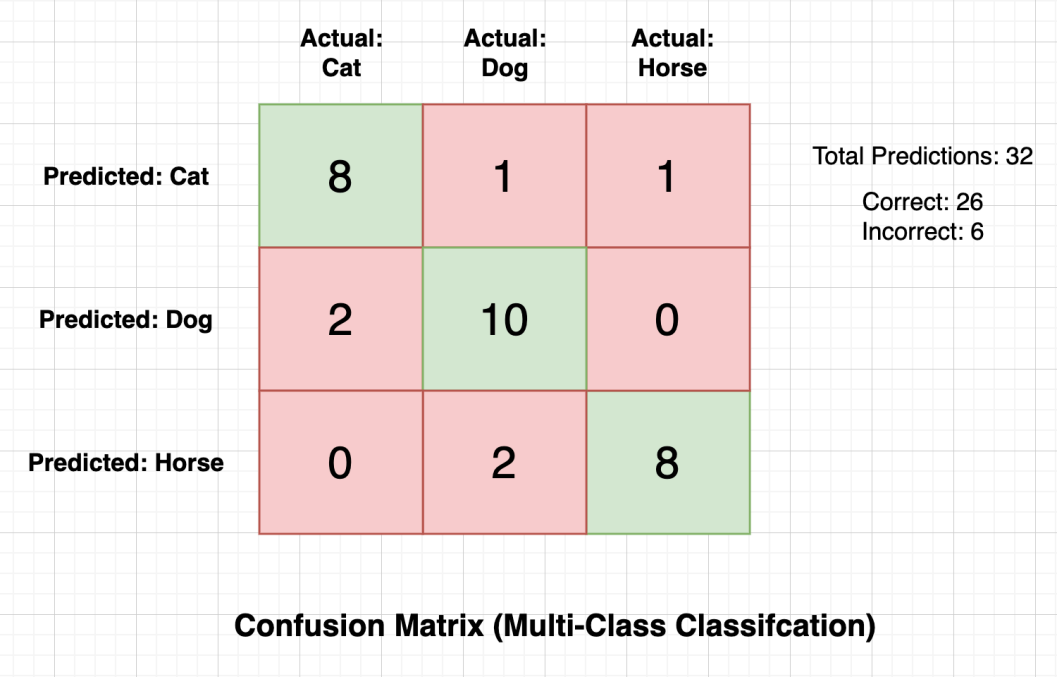
So if you have n classes, you get an $n \times n$ **matrix**.

Now here's the key structure you need to understand:

- The **diagonal elements:** These are always the **correct predictions.** In other words, the **True Positives for each class**

- Everything **outside the diagonal:** These are mistakes. Misclassifications, confusion between classes, exactly what the name suggests

Here is what the multi-class confusion matrix looks like:



- The green diagonal cells represent the **26 correct predictions** where the model's classification matched the actual animal category.
- The red off-diagonal cells highlight the **6 incorrect predictions**, showing exactly where the model misidentified one animal as another.

But now comes the tricky part. In binary classification, TP, FP, FN, TN are obvious. In multi-class, they are **relative**. Because now, you have to look at the matrix from the perspective of **one class at a time**.

Let's say we focus on a specific class: **class** k

Now we redefine everything *from its perspective*:

- **True Positives (TP_k):** This is the easiest one. It's simply the diagonal element:

$$TP_k = M[k, k]$$

- **False Positives (FP_k):** These are cases where the model predicted class k, but the actual class was something else.

So we take the **entire column** k, and exclude the diagonal:

$$FP_k = (\sum column_k) - TP_k$$

- **False Negatives (FN_k):** These are cases where the actual class is k, but the model predicted something else.

So we take the **entire row** k, and exclude the diagonal:

$$FN_k = (\sum row_k) - TP_k$$

- **True Negatives (TN_k):** This is everything else. All samples that are:
  - Not actually class k
  - And not predicted as class k

So the easiest way to compute it:

$$TN_k = Total_{samples} - (TP_k + FP_k + FN_k)$$

And if you notice something here, FP and FN are no longer single mistakes. They are **aggregations of all the ways the model gets class k wrong**.

And that's the shift I want you to understand: In multi-class problems, you don't evaluate the model *once,* you now evaluate it **per class**.

Each class gets its own: Precision; Recall; Specificity, and F1-score

So the confusion matrix is no longer just a table. It becomes a **lens**. A way to zoom in and analyze the model's behavior for *each individual class*.

Because sometimes your model is excellent at one class, and completely fails at another. And without this matrix? You would never know.

Now, this is where multi-class classification becomes *real*. We're no longer just thinking in theory—we're actually building everything from scratch.

So let me walk you through what each part is doing, because nothing here is random.

```
import numpy as np

true_value = np.array([
    'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat',
'Cat', 'Cat',
    'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog',
'Dog', 'Dog', 'Dog', 'Dog', 'Dog',
    'Horse', 'Horse', 'Horse', 'Horse', 'Horse', 'Horse',
'Horse', 'Horse', 'Horse'
])
predict_value = np.array([
    'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat',
'Dog', 'Dog',
    'Cat', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog',
'Dog', 'Dog', 'Dog', 'Horse', 'Horse',
    'Cat', 'Horse', 'Horse', 'Horse', 'Horse', 'Horse', 'Ho
rse', 'Horse', 'Horse'
])

# Total samples
print(f"Total Samples: {len(true_value)}")
```

Total Samples: 32

```
unique_labels = np.unique(true_value).tolist()
# Replace the labels
for i in range(len(unique_labels)):
    true_value[true_value == unique_labels[i]] = i
    predict_value[predict_value == unique_labels[i]] = i

true_value = np.array(true_value, dtype=int)
predict_value = np.array(predict_value, dtype=int)
```

We start with labels like **“Cat”**, **“Dog”**, **“Horse”**. But remember, models don’t understand text.

So we convert them into numbers: $Cat \to 0; Dog \to 1; Horse \to 2$

This is just a mapping step—nothing fancy, but absolutely necessary.

This is where we construct the matrix itself:

```
def multi_confusion_matrix(true_value, predict_value, label
s):
  confusion_matrix = np.zeros((len(labels), len(labels)))
  tp = []
  for j in range(len(unique_labels)):
      tp_count = 0
      for k in range(len(true_value)):
          # Catch the true positive
          if true_value[k] == j and predict_value[k] == j:
              tp_count += 1  # Count and then append
          # Finally, count the remaining entries
          elif true_value[k] == j and predict_value[k] !=
j:
              confusion_matrix[true_value[k], predict_value
[k]] += 1
      # Substitute the tp to the diagonal entry
      confusion_matrix[j, j] = tp_count

  return confusion_matrix
```

Think back to the binary case, we were counting TP, TN, FP, FN manually. Here, we do the same thing—but across multiple classes.

What this function does:

- It initializes an `n × n` **matrix** (based on number of classes)
- Then for each class `j`:
  - It counts **True Positives** → when `true_value == j and predict_value == j`
  - For all other cases where the true class is `j` but prediction is different → it fills the **off-diagonal entries**

So in the end:

- The **diagonal =** TP **for each class**
- The **off-diagonal = misclassifications**

That's exactly the structure we discussed earlier.

This next function is the heart of multi-class evaluation:

```
def get_components(confusion_matrix, k):
    tp = confusion_matrix[k, k]
    fp = confusion_matrix[:, k].sum() - tp
    fn = confusion_matrix[k, :].sum() - tp
    tn = confusion_matrix.sum() - (tp + fp + fn)
    return tp, fp, fn, tn
```

Remember what I said: In multi-class, TP, FP, FN, TN are defined **per class**

So for a given class `k`:

- `tp` → `confusion_matrix[k, k]`
- `fp` → sum of column `k` minus `tp`
- `fn` → sum of row `k` minus `tp`
- `tn` → everything else

This function is literally implementing those formulas directly. No shortcuts. Just logic.

```
def evaluation(labels):
  confusion_matrix = multi_confusion_matrix(true_value, pre
dict_value, labels)
  for i in range(len(labels)): # Iterate over integer indic
es
      print("Label", labels[i]) # Use the index to get the
string label for printing
      tp, fp, fn, tn = get_components(confusion_matrix, i)
      accuracy = (tp + tn) / (tp + tn + fp + fn)
      precision = tp / (tp + fp)
      recall = tp / (tp + fn)
      specificity = tn / (tn + fp)
      f1_score = (2 * recall * precision) / (recall + preci
sion)
```

```
        print(f"- TP: {tp}; FP: {fp}; FN: {fn}; TN: {tn}")
        print(f"- Accuracy: {accuracy}")
        print(f"- Precision: {precision}")
        print(f"- Recall: {recall}")
        print(f"- Specificity: {specificity}")
        print(f"- F1 Score: {f1_score}")
```

For each class:

- We extract TP, FP, FN, TN
- Then compute: Accuracy; Precision; Recall; Specificity; and F1-score

And this is the key difference from binary classification: We are not getting **one score** → We are getting **a score per class**

Execute the functions:

```
multi_confusion_matrix(true_value, predict_value, unique_la
bels)
evaluation(unique_labels)
```

```
array([[ 8.,  2.,  0.],
       [ 1., 10.,  2.],
       [ 1.,  0.,  8.]])
```

Label Cat

- TP: 8.0; FP: 2.0; FN: 2.0; TN: 20.0
- Accuracy: 0.875
- Precision: 0.8
- Recall: 0.8
- Specificity: 0.9090909090909091
- F1 Score: 0.8000000000000002

Label Dog

- TP: 10.0; FP: 2.0; FN: 3.0; TN: 17.0
- Accuracy: 0.84375
- Precision: 0.8333333333333334
- Recall: 0.7692307692307693
- Specificity: 0.8947368421052632
- F1 Score: 0.8

Label Horse

- TP: 8.0; FP: 2.0; FN: 1.0; TN: 21.0
- Accuracy: 0.90625
- Precision: 0.8
- Recall: 0.8888888888888888
- Specificity: 0.9130434782608695
- F1 Score: 0.8421052631578948

The confusion matrix tells an interesting story:

- **Cat** → sometimes confused with Dog
- **Dog** → confused with both Cat and Horse
- **Horse** → mostly correct, very few mistakes

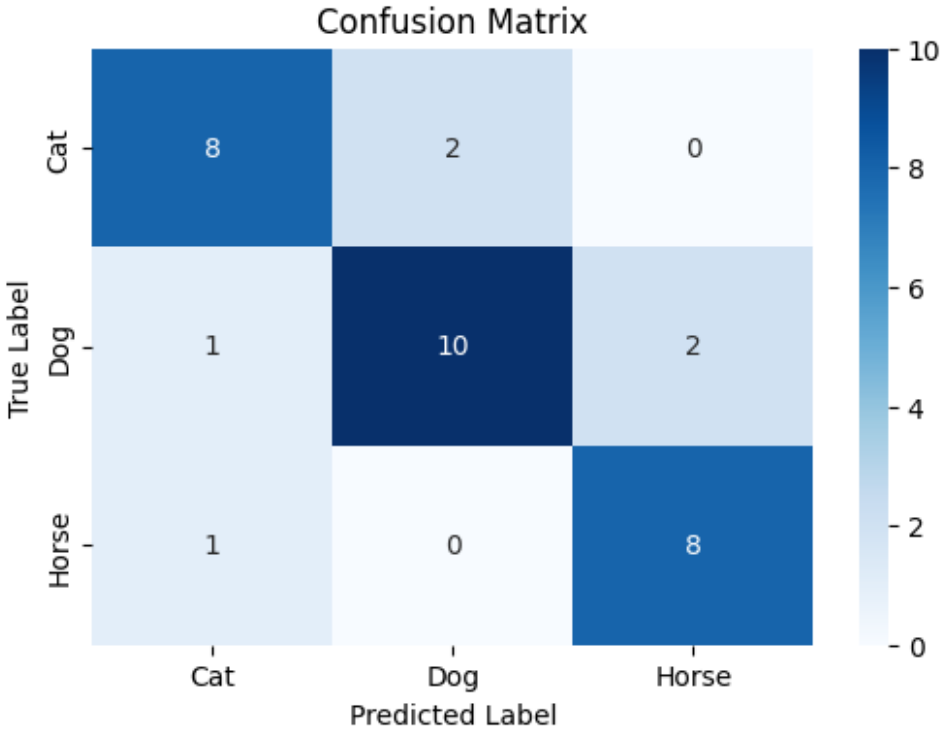
And when you look at the metrics:

- Cat → balanced performance ($Precision = Recall = 0.8$)
- Dog → slightly weaker recall (misses some Dogs)
- Horse → strongest recall (~0.89), very reliable

Plot the confusion matrix:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot
confusion_matrix = multi_confusion_matrix(true_value, predi
ct_value, unique_labels)
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix, annot=True, cmap="Blues",
            xticklabels=unique_labels, yticklabels=unique_l
abels)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



4. Confusion Matrix - Macro Metrics

So far, we've been evaluating **each class individually**. But sometimes, you don't want separate scores. You want a **single number** that summarizes the model's performance across all classes. That's where **Macro metrics** come in.

The idea is actually very simple:

- **Macro Precision** → average of Precision across all classes
- **Macro Recall** → average of Recall across all classes
- **Macro Specificity** → average of **Specificity** across all classes
- **Macro F1-Score** → average of F1-scores across all classes

So instead of weighting by how many samples each class has. We treat every class **equally**. That means a small class matters just as much as a large class. You get a more **balanced view** of performance

This is important. Because sometimes a model looks "good" overall, but completely fails on a minority class. Macro metrics help expose that:

```
# Find the macro metrics
def macro_evaluation(labels):
  confusion_matrix = multi_confusion_matrix(true_value, pre
dict_value, labels)
  precision = []
  recall = []
  specificity = []
  f1_score = []
  for i in range(len(labels)): # Iterate over integer indic
es
      tp, fp, fn, tn = get_components(confusion_matrix, i)
      accuracy = (tp + tn) / (tp + tn + fp + fn)
      precision.append(tp / (tp + fp))
      recall.append(tp / (tp + fn))
      specificity.append(tn / (tn + fp))
      f1_score.append((2 * recall[i] * precision[i]) / (rec
all[i] + precision[i]))
  # Find Macro
  precision = np.mean(precision)
  recall = np.mean(recall)
  specificity = np.mean(specificity)
  f1_score = np.mean(f1_score)
  return f"Macro Precision: {precision},\nMacro Recall: {re
call},\nMacro Specificity: {specificity},\nMacro F1-Score:
{f1_score}"


print(macro_evaluation(unique_labels))
```

```
Macro Precision: 0.6022727272727273,
Macro Recall: 0.5833333333333334,
Macro Specificity: 0.5833333333333334,
Macro F1-Score: 0.5248868778280543
```

5. Confusion Matrix with Python

The Python Scikit-Learn library allows you to construct a confusion matrix directly:

```
import numpy as np
from sklearn.metrics import confusion_matrix, classificatio
n_report, accuracy_score

true_value = np.array([
    'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat',
'Cat', 'Cat',
    'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog',
'Dog', 'Dog', 'Dog', 'Dog', 'Dog',
    'Horse', 'Horse', 'Horse', 'Horse', 'Horse', 'Horse',
'Horse', 'Horse', 'Horse', 'Horse'
])
predict_value = np.array([
    'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat', 'Cat',
'Dog', 'Dog',
    'Cat', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Dog',
'Dog', 'Dog', 'Dog', 'Horse', 'Horse',
    'Cat', 'Horse', 'Horse', 'Horse', 'Horse', 'Horse', 'Ho
rse', 'Horse', 'Horse'
])

conf_matrix = confusion_matrix(true_value, predict_value, l
abels=['Cat', 'Dog', 'Horse'])
print("Confusion Matrix:\n", conf_matrix)

accuracy = accuracy_score(true_value, predict_value)
print("Accuracy:", accuracy)
classification_rep = classification_report(true_value, pred
ict_value)
print("Classification Report:\n", classification_rep)
```

Confusion Matrix:
[[8 2 0]
[1 10 2]
[1 0 8]]
Accuracy: 0.8125
Classification Report:
precision recall f1-score support
Cat 0.80 0.80 0.80 10

Dog	0.83	0.77	0.80	13
Horse	0.80	0.89	0.84	9
accuracy			0.81	32
macro avg	0.81	0.82	0.81	32
weighted avg	0.81	0.81	0.81	32

```
import numpy as np
from sklearn.metrics import confusion_matrix, accuracy_scor
e, precision_score, recall_score, f1_score

# A is for positive and B is for Negative
true_value = np.array(['Not Spam', 'Spam', 'Not Spam', 'Spa
m', 'Spam', 'Spam',
                       'Not Spam', 'Not Spam', 'Not Spam',
'Spam',
                       'Spam', 'Not Spam', 'Spam', 'Spam',
'Spam'])
predict_value = np.array(['Not Spam', 'Not Spam', 'Spam',
'Not Spam',
                          'Spam', 'Spam', 'Not Spam', 'Not
Spam',
                          'Not Spam', 'Not Spam', 'Not Spa
m', 'Not Spam', 'Not Spam', 'Not Spam', 'Spam'])

# Confusion Matrix
conf_matrix = confusion_matrix(true_value, predict_value, l
abels=['Spam', 'Not Spam'])

print("Confusion Matrix for Not Spam:\n", conf_matrix)

accuracy = accuracy_score(true_value, predict_value)
print("Accuracy:", accuracy)
```

Confusion Matrix:
[[3 6]
[1 5]]
Accuracy: 0.5333333333333333

```
print("\nLabel: Spam")
precision = precision_score(true_value, predict_value, pos_
label='Spam')
print("Precision:", precision)
recall = recall_score(true_value, predict_value, pos_label
='Spam')
print("Recall:", recall)
f1 = f1_score(true_value, predict_value, pos_label='Spam')
print("F1 Score:", f1)

print("\nLabel: Not Spam")
precision = precision_score(true_value, predict_value, pos_
label='Not Spam')
print("Precision:", precision)
recall = recall_score(true_value, predict_value, pos_label
='Not Spam')
print("Recall:", recall)
f1 = f1_score(true_value, predict_value, pos_label='Not Spa
m')
print("F1 Score:", f1)
```

```
Label: Spam
Precision: 0.75
Recall: 0.3333333333333333
F1 Score: 0.46153846153846156

Label: Not Spam
Precision: 0.45454545454545453
Recall: 0.8333333333333334
F1 Score: 0.5882352941176471
```

Let's look at this from a slightly different angle. Up until now, we've been very consistent: We pick a **positive class,** and compute everything based on that.

In this example, that class is **"Spam"**. So when we say:

- TP → correctly predicted Spam
- TN → correctly predicted as not Spam

- FP → predicted Spam but actually Not Spam
- FN → missed Spam

Everything revolves around that choice. But here's something many learners don't realize: You can flip the perspective. You can treat **"Not Spam"** as the positive class. And when you do that, everything shifts.

Look at the results:

For **Spam**:

- $Precision = 0.75$
- $Recall = 0.33$

For **Not Spam**:

- $Precision \approx 0.45$
- $Recall \approx 0.83$

Same model. Same predictions. Completely different interpretation.

Why? Because when you switch the "positive label":

- TP **becomes** TN
- TN **becomes** TP
- FP **becomes** FN
- FN **becomes** FP

You are literally **rotating your perspective** of the confusion matrix.

And this is a very important idea. Because in practice, we usually focus on **one class** (the "positive" one). Especially when that class is *important:* Spam detection; Disease detection; Fraud detection. But that doesn't mean the other class is irrelevant.

In this example:

- The model is **good at detecting Not Spam** (high recall ~0.83)
- But **bad at detecting Spam** (low recall ~0.33)

So what does that mean?

- The model is *playing it safe*

- It prefers to say “Not Spam”
- And as a result… it misses a lot of actual Spam

And this brings us back to a deeper insight:

> ***A Confusion Matrix is not fixed. It depends on what you choose to care about.***

So when evaluating a model, always ask:

- What is my **positive class**?
- What kind of errors matter more?
  - False positives?
  - False negatives?

Because the exact same model, can look **good or bad** depending on that choice. Not just calculating metrics, but understanding **what they actually mean**.

One last detail that can easily confuse people—and ironically, it’s about how the **Confusion Matrix itself is implemented using Scikit-Learn**. When you use **Scikit-Learn**, the convention is:

- **Rows → Actual labels**
- **Columns → Predicted labels**

So this confusion matrix…

```
[3 6]
[1 5]]
```

should be read as:

- Row 0 → Actual **Spam**
- Row 1 → Actual **Not Spam**
- Column 0 → Predicted **Spam**
- Column 1 → Predicted **Not Spam**

Now here’s the key point: In **Scikit-Learn**, the order of classes is determined by the labels parameter.

In this case: `labels = ['Spam', 'Not Spam']`

So: Index 0 → Spam; and Index 1 → Not Spam

That means the matrix is interpreted as:

```
           Predicted
           Spam   Not Spam
Actual
Spam        3        6
Not Spam    1        5
```

Now let's map this to TP, FP, FN, TN **for the class "Spam"**:

- $TP = 3$ → Actual Spam, Predicted Spam
- $FN = 6$ → Actual Spam, Predicted Not Spam
- $FP = 1$ → Actual Not Spam, Predicted Spam
- $TN = 5$ → Actual Not Spam, Predicted Not Spam

So in matrix form:

```
[ TP   FN ]
[ FP   TN ]
```

And this leads to an important takeaway:

- Scikit-Learn doesn't "magically know" your positive class
- It simply follows the **order you give in labels**

But when you use functions like: `precision_score; recall_score; f1_score`; By default, it uses the first label it encounters. Unless we explicitly specify one. For instance: `pos_label='Spam'`

That's when you are telling the model: *"This is the class I care about. Treat this as positive".* So don't get confused:

- The **matrix layout** is fixed (rows = actual, columns = predicted)
- But $TP/FP/FN/TN$ **depend on perspective**

And this is exactly why beginners get tripped up: Same matrix. Different interpretations.

Once you understand this, you're no longer just reading a Confusion Matrix—You're **controlling how it is interpreted**.

If you want to switch perspective and look at **"Not Spam"** as the positive class, you don't need to rebuild everything. You already have the confusion matrix. You just need to **rearrange it**.

Remember the current structure (Scikit-Learn):

```
[[TP, FN],
 [FP, TN]]
```

This is from the perspective of **Spam** as the positive class.

To flip the perspective to **Not Spam**, you essentially:

- Swap $TP \leftrightarrow TN$
- Swap $FP \leftrightarrow FN$

So the matrix becomes:

```
[[TN, FP],
 [FN, TP]]
```

And using NumPy, this is actually very clean:

```
flipped_matrix = conf_matrix[[1, 0]][:, [1, 0]]
```

You don't need to recompute TP, FP, FN, TN from scratch. The information is already there, you just need to **look at it from a different angle**.

Because at the end of the day: A Confusion Matrix is not just data. It's a **perspective tool**. Change the perspective, and you change the story your model is telling.

Summary

Let's step back and connect everything. When you first learn Machine Learning, it's very easy to think that evaluation is just about **Accuracy**. A single number. A quick answer. Something that tells you: *"Is my model good or not?"*

But now you know, it's not that simple. The moment the **Confusion Matrix is introduced**, everything changes. Instead of one number, you get structure:

- What the model gets right
- What it gets wrong
- And *how* it gets it wrong

From that structure, you derive:

- Precision → how reliable predictions are
- Recall → how much you are missing
- Specificity → how well you avoid false alarms
- F1-score → how balanced your model is

And in multi-class problems:

- You analyze **each class individually**
- Then optionally summarize using **macro metrics**

So evaluation is no longer just a final step. It becomes part of your **understanding** of the model.

Because two models with the same accuracy can behave in completely different ways.

And that brings us back to the core idea Iris has been building all along:

> Machine Learning is not just about building models. It's about **understanding their behavior**.

And the Confusion Matrix? It's the tool that lets you see that behavior clearly. So if there's one thing you should take away from this entire section, it's this:

> **The Confusion Matrix is what we need to calculate—and truly understand—evaluation metrics in classification problems.**

References

https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/

https://www.ibm.com/think/topics/confusion-matrix

https://www.datacamp.com/tutorial/what-is-a-confusion-matrix-in-machine-learning

https://www.youtube.com/watch?v=Kdsp6soqA7o

Next Section:

- 5. Covariance Matrix and Correlation Matrix